



Paradigmas de Programación (EIF-400) Caso de FP-OOP en Java Parte-C

CARLOS LORÍA-SÁENZ LORACARLOS@GMAIL.COM

AGOSTO-SETIEMBRE 2025

EIF/UNA

Objetivos

2

- ▶ Opciones nuevas para data types y pattern-matching
- ▶ Opciones para interpolación de hileras y multilíneas

Objetivos Específicos

3

- ▶ Data types con `record`
- ▶ sealed classes
- ▶ switch pattern matching
- ▶ `instanceof` pattern-matching
- ▶ when guardas
- ▶ `yield`

record

4

- ▶ Forma concisa de declarar una clase de objetos que principalmente son datos sin demasiada lógica
- ▶ Adecuados para POJOs, DTOs
- ▶ El compilador genera mucho del código automáticamente
 - ▶ Getters
 - ▶ toString
 - ▶ Equal estructural
- ▶ Son sellados (final)
- ▶ Se pueden customizar

Ejemplo

5

```
jshell> record Person(String name, int age){}  
| modified record Person
```

```
jshell> var juan = new Person("Juan Perez", 23)  
juan ==> Person[name=Juan Perez, age=23]
```

```
jshell> juan.name()  
$9 ==> "Juan Perez"
```

getters

```
jshell> juan.age()  
$10 ==> 23
```

```
jshell> juan  
juan ==> Person[name=Juan Perez, age=23]
```

toString

```
jshell> var another_juan = new Person("Juan Perez", 23)  
another_juan ==> Person[name=Juan Perez, age=23]
```

```
jshell> juan.equals(another_juan)  
$13 ==> true
```

equals

```
jshell> |
```

Constructor Canónico

6

► Forma compacta

```
jshell> record Person(String name, int age){  
...>     public Person{  
...>         if (name == null || age < 0)  
...>             throw new IllegalArgumentException("Person invalid name or age");  
...>     }  
...> }  
| created record Person
```

Customizando un accesor

7

```
jshell> record Person(String name, int age){
...>     public Person{
...>         if (name == null || age < 0)
...>             throw new IllegalArgumentException("Person invalid name or age");
...>     }
...>     public String name(){
...>         System.out.println("Accessing name");
...>         return name;
...>     }
...> }
| modified record Person
```

```
jshell> var juan =new Person("Juan Perez", 23)
juan ==> Person[name=Juan Perez, age=23]

jshell> juan.name()
Accessing name
$4 ==> "Juan Perez"

jshell> |
```


Campo/método estático

8

```
jshell> record Person(String name, int age){
...>     public Person{
...>         if (name == null || age < 0)
...>             throw new IllegalArgumentException("Person invalid name or age");
...>         persons++;
...>     }
...>     public String name(){
...>         System.out.println("Accessing name");
...>         return name;
...>     }
...>     private static int persons = 0;
...>     static public int howManyPersons(){
...>         return persons;
...>     }
...> }
| modified record Person

jshell> var juan =new Person("Juan Perez", 23)
juan ==> Person[name=Juan Perez, age=23]

jshell> var ana =new Person("Ana Perez", 23)
ana ==> Person[name=Ana Perez, age=23]

jshell> Person.howManyPersons()
$16 ==> 2
```


Generics

```
jshell> record Tuple<X, Y>(X x, Y y){}  
|   created record Tuple
```

```
jshell> var t = new Tuple<String, Integer>("Juan", 23)  
t ==> Tuple[x=Juan, y=23]
```

```
jshell> t.x() + ", " + t.y()  
$21 ==> "Juan, 23"
```

```
jshell> |
```

record e Interfaces

10

- Un record puede implementar una interfaz

```
jshell> interface Animal{ String name();}  
|   created interface Animal  
  
jshell> record Dog(String name) implements Animal{}  
|   created record Dog  
  
jshell> var fido = new Dog("Fido")  
fido ==> Dog[name=Fido]  
  
jshell> if (fido instanceof Animal) println("Woof")  
Woof  
  
jshell> |
```

Clases selladas (sealed)

11

- ▶ Clases que limitan cuáles clases las pueden extender
- ▶ Controlar casos permitidos de clases derivadas en casos que se conozcan exhaustivamente
- ▶ Eso le permite al compilador, por ejemplo, determinar exhaustividad de casos en un switch o evitar castings inválidos en tiempo estático
- ▶ Son como una generalización de final classes
- ▶ La O de SOLID puede ser demasiado abierta y confundir en casos que el dominio es bien conocido y muy estable: Por ejemplo, la sintaxis de un lenguaje de programación como Java. Pensar en el Patrón Command

Ejemplo (ASTs)

12

```
jshell> sealed interface Expr<T>{  
...>     record ConstantInt(Integer value) implements Expr<Integer>{  
...>         public Integer eval(){return value;}  
...>     }  
...>     record AddInt(Expr<Integer> x, Expr<Integer> y) implements Expr<Integer>{  
...>         public Integer eval(){  
...>             return x.eval() + y.eval();  
...>         }  
...>     }  
...>     T eval();  
...> }
```

| created interface Expr

```
jshell> var c666 = new Expr.ConstantInt(666)  
c666 ==> ConstantInt[value=666]
```

```
jshell> var c999 = new Expr.ConstantInt(999)  
c999 ==> ConstantInt[value=999]
```

```
jshell> var a = new Expr.AddInt(c666, c999)  
a ==> AddInt[x=ConstantInt[value=666], y=ConstantInt[value=999]]
```

```
jshell> a.eval()  
$5 ==> 1665
```

```
jshell> 666 + 999  
$6 ==> 1665
```

...AST switch expression

13

```
JSHELL:jshell --enable-preview  
| Welcome to JShell -- Version 17.0.1  
| For an introduction type: /help intro  
  
jshell> |
```

Requiere en versiones < 21 usar la opción `--enable-preview`

```
jshell> String ASTtoString(Expr<Integer> e){  
...>     return switch (e){  
...>         case Expr.ConstantInt c -> String.format("%d", c.value());  
...>         case Expr.AddInt a -> String.format("%s + %s", ASTtoString(a.x()), ASTtoString(a.y()));  
...>     };  
...> }  
| created method ASTtoString(Expr<Integer>)  
  
jshell> var a = new Expr.AddInt(new Expr.ConstantInt(666), new Expr.ConstantInt(777))  
a ==> AddInt[x=ConstantInt[value=666], y=ConstantInt[value=777]]  
  
jshell> ASTtoString(a)  
$4 ==> "666 + 777"  
  
jshell> |
```

...AST switch case: yield

14

```
jshell> String ASTtoString(Expr<Integer> e){  
...>     return switch (e){  
...>         case Expr.ConstantInt c :  
...>             println("Seeing Constant");  
...>             yield String.format("%d", c.value());  
...>         case Expr.AddInt a :  
...>             println("Seeing Add");  
...>             yield String.format("%s + %s", ASTtoString(a.x()), ASTtoString(a.y()));  
...>     };  
...> }  
| created method ASTtoString(Expr<Integer>)
```

```
jshell> var a = new Expr.AddInt(new Expr.ConstantInt(666), new Expr.ConstantInt(777))  
a ==> AddInt[x=ConstantInt[value=666], y=ConstantInt[value=777]]
```

```
jshell> ASTtoString(a)  
Seeing Add  
Seeing Constant  
Seeing Constant  
$4 ==> "666 + 777"
```

```
jshell> |
```

Bloques de texto

15

- ▶ Facilidad para tener hileras multilínea
- ▶ Se usan triple doble comillas
- ▶ Debe haber un cambio de línea seguido de la apertura del bloque
- ▶ **Margen incidental**: El primer (es decir, menor) margen de espacios o el que sobra en una línea (llamado incidental) es ignorado al procesar el bloque. Se conserva solo el espacio esencial.
- ▶ Esto permite una mejor que un cambio en el margen afecte el contenido

Ejemplo

16

```
jshell> String genPageWithMessage(String msg){  
...>     String page = ""  
...>         <html>  
...>             <h1>Hello %s!</h1>  
...>         </html>"";  
...>     return String.format(page, msg);  
...> }  
| created method genPageWithMessage(String)
```

```
jshell> genPageWithMessage(" World")  
$6 ==> "<html>\n    <h1>Hello  World!</h1>\n</html>"
```

```
jshell> |
```

Ejemplo:

- Si se quiere “conservar” el margen incidental inicial se cierra el bloque como el primer carácter de la línea

```
jshell> String genPageWithMessage(String msg){  
...>     String page = ""  
...>         <html>  
...>             <h1>Hello %s!</h1>  
...>         </html>  
...>     "";  
...>     return String.format(page, msg);  
...> }  
| modified method genPageWithMessage(String)  
  
jshell> genPageWithMessage(" World")  
$8 ==> "         <html>\n                <h1>Hello  World!</h1>\n                </html>\n"
```

Blancos sobrantes

- ▶ Los blancos al final de cada línea son considerados incidentales.
- ▶ Si se quieren conservar hay varias técnicas.
- ▶ Por ejemplo: se puede usar alguna marca (ejemplo |) que sirva de frontera y se elimine

```
jshell> var s = ""  
...>     siguen blancos que quiero conservar |  
...>     siguen blancos que no quiero conservar  
...> ""  
s ==> "    siguen blancos que quiero conservar | \n ... que no quiero conservar \n"  
  
jshell> s.replace("| \n", "\n")  
$15 ==> "    siguen blancos que quiero conservar  \n    siguen blancos que no quiero conservar \n"
```

■ Blancos esenciales

■ Blancos incidentales

Detalles adicionales

19

- ▶ Más consideraciones sobre y soluciones son ilustrados en la [referencia](#). Incluye Guía de estilos recomendados

Ejemplo

20

► Ver PatternsExamples.java

```
D:\UNA\Paradigmas\CursoLenguajesDeProgramación2024\04-Clases\02-FP\FP-2-Java\work\PatternMatching>javac -d classes src\PatternsExamples.java
D:\UNA\Paradigmas\CursoLenguajesDeProgramación2024\04-Clases\02-FP\FP-2-Java\work\PatternMatching>java -cp classes PatternsExamples
*** Patterns Examples ***
Large circle with radius 5.0
D:\UNA\Paradigmas\CursoLenguajesDeProgramación2024\04-Clases\02-FP\FP-2-Java\work\PatternMatching>
```